SMC Municipal Health Command Dashboard for Arogya-SMC: Complete Design Document

1. Overview

The **Arogya-SMC Municipal Health Command Dashboard** is the central nervous system of the entire platform – a comprehensive web application designed for Municipal Health Officers (MHOs), epidemiologists, and SMC administrators. It provides real-time situational awareness across Solapur's healthcare ecosystem, enabling evidence-based decision-making, outbreak response, and resource optimization .

Key Objectives:

- Provide a unified, real-time view of ward-level health intelligence
- Enable early outbreak detection through predictive analytics
- Monitor city-wide hospital capacity and resource stress
- Manage and disseminate public health advisories
- Facilitate communication with field staff (ASHAs, hospitals)
- Generate actionable insights for policy planning

---

2. Technology Stack (100% Free)

Core Framework

Component	Technology	Justification	Cost
Framework	Next.js 15+ (App Router)	React-based, server-side rendering, built-in API routes, excellent performance for data-heavy dashboards	Free
Language	TypeScript 5+	Type safety, better developer experience	Free
Styling	Tailwind CSS 4+	Utility-first, rapid UI development, responsive design	Free
UI Components	shadcn/ui	Accessible, customizable component library built on Radix UI	Free

Data Visualization

Component	Technology	Purpose
Maps	Leaflet + react-leaflet	Ward-level heatmaps, facility locations, outbreak zones
Charts	Recharts / Tremor	Time-series trends, comparisons, resource utilization
Tables	TanStack Table	Sortable, filterable data tables for alerts and reports

State Management & Data Fetching

Component	Technology	Purpose
Server State	TanStack Query (React Query) v5	Data fetching, caching, real-time updates
Real-time Updates	Server-Sent Events / WebSockets	Live alert notifications
HTTP Client	Axios	API requests with interceptors

Authentication & Security (Prototype)

Component	Technology	Purpose
Authentication	NextAuth.js (Auth.js)	Email/password with role-based access (MHO, Analyst, Admin)
JWT Handling	js-cookie + jose	Token management

Development Tools

Tool	Purpose
ESLint	Code linting
Prettier	Code formatting
Husky	Git hooks
Postman	API testing

3. Understanding Municipal Health Dashboard Requirements

Industry Standards & Best Practices

Based on analysis of existing health command centres and surveillance systems :

Core Functions of a Health Command Centre:

- **Real-time data aggregation** from hospitals, labs, and field workers
- **Geospatial visualization** of disease trends and resource distribution
- **Automated alerting** for outbreak detection and resource stress
- **Communication module** for issuing directives to field staff
- **Analytics engine** for trend analysis and forecasting
- **Training and review** capabilities for health personnel

Key User Types:

User	Role	Permissions
Municipal Health Officer (MHO)	Overall oversight	Full access, can issue advisories

User	Role	Permissions
Epidemiologist	Disease surveillance	View all data, run analyses
Data Analyst	Report generation	View aggregated data, export reports
Emergency Coordinator	Crisis response	Full access during emergencies

Data Integration Points

The dashboard must integrate data from multiple sources :

Source	Data Type	Update Frequency
ASHA App	Syndromic reports, risk flags	Daily / Real-time
Hospital Portal	Bed capacity, ICU, oxygen, disease counts	Daily / Real-time
Labs (future)	Test results, confirmations	As available
Weather/Environment	Rainfall, temperature (external)	Hourly
IDSP (future)	National disease surveillance data	Periodic

4. Features & Functions

Core Features (Must-Have for March 23)

Feature	Description	Priority
F1: Secure Login	Role-based authentication for different user types	HIGH
F2: Executive Dashboard	City-level summary with KPIs (active alerts, high-risk wards, bed availability)	HIGH
F3: Ward-Level Heatmap	Interactive map showing disease risk by ward with color coding	HIGH
F4: Time-Series Charts	Trends for fever, cough, diarrhea over selectable periods	HIGH
F5: Facility Resource Monitor	Real-time view of hospital bed/ICU/ventilator/oxygen status across city	HIGH
F6: Alert Management	List of active alerts with severity, location, and action buttons	HIGH
F7: Advisory Creation	Form to create and publish public health advisories (with ward targeting)	HIGH
F8: Communication Module	Send push notifications to ASHA workers in specific wards	MEDIUM
F9: Data Export	Export reports as CSV/PDF for documentation	MEDIUM
F10: User Management	Basic user list with roles (for prototype, hardcoded)	LOW

Secondary Features (Post-Prototype)

Feature	Description
Predictive Analytics	ML-based outbreak forecasting
Training Module	Video conferencing and training materials for field staff
Helpline Integration	104 helpline call data and patient follow-up
Wastewater Surveillance	Integration with environmental monitoring data
Interoperability	Connection to DHIS2 or other national systems

5. Data Model

```
// Dashboard User
interface DashboardUser {
  id: string;
  name: string;
  email: string;
  role: 'mho' | 'epidemiologist' | 'analyst' | 'emergency';
  lastLogin?: string;
}

// Ward Summary (for dashboard)
interface WardSummary {
  wardCode: string;
  wardName: string;
  riskScore: number; // 0-1 calculated from multiple indicators
  population?: number;

  // Syndromic counts (last 24h)
  syndromic: {
    fever: number;
    cough: number;
    diarrhea: number;
    jaundice: number;
  };

  // Resource status
  facilities: {
    total: number;
    bedsAvailable: number;
    icuAvailable: number;
    oxygenAvailable: boolean;
  };

  // Alerts
  activeAlerts: number;
}
```

```typescript
// Dashboard Summary Response
interface DashboardSummary {
  timestamp: string;
  cityLevel: {
    totalWards: number;
    highRiskWards: number;
    activeAlerts: number;
    facilitiesReporting: number;
    totalBedsAvailable: number;
    totalIcuAvailable: number;
  };
  wards: WardSummary[];
}

// Time-Series Data Point
interface TrendDataPoint {
  date: string; // YYYY-MM-DD
  fever: number;
  cough: number;
  diarrhea: number;
  jaundice: number;
  total: number;
}

// Alert
interface DashboardAlert {
  id: string;
  type: 'outbreak' | 'resource' | 'info' | 'warning';
  severity: 'low' | 'medium' | 'high' | 'critical';
  wardCode: string;
  wardName: string;
  title: string;
  description: string;
  generatedAt: string;
  acknowledgedAt?: string;
  acknowledgedBy?: string;
  resolvedAt?: string;
  resolvedBy?: string;
  status: 'active' | 'acknowledged' | 'resolved';
}

// Public Advisory (created by MHO)
interface PublicAdvisory {
  id: string;
  title: string;
  description: string;
  severity: 'low' | 'medium' | 'high';
  wardCode?: string; // null = city-wide
  publishedAt: string;
  publishedBy: string;
  expiresAt?: string;
  languageVersions?: {
```

```
    mr?: { title: string; description: string };
    hi?: { title: string; description: string };
  };
}
```

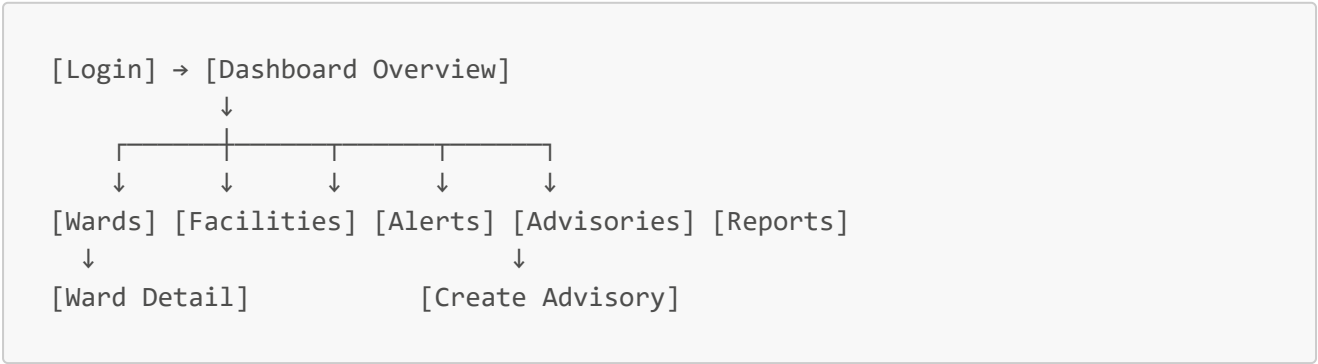
---

6. Pages / Screens Design

Screen Overview

Screen	Route	Purpose
Login	`/login`	Authentication
Dashboard Overview	`/dashboard`	City-level KPIs, alerts summary, trend preview
Ward Analytics	`/wards`	Detailed ward-level heatmap and data table
Ward Detail	`/wards/[code]`	Drill-down into specific ward
Facility Monitor	`/facilities`	Real-time hospital capacity across city
Alerts	`/alerts`	Full alert management interface
Advisories	`/advisories`	Create and manage public advisories
Communication	`/communicate`	Send notifications to ASHA workers
Reports	`/reports`	Generate and export reports
Settings	`/settings`	User profile, system settings

Screen Flow Diagram



---

7. Detailed Screen Specifications

Screen 1: Login (`/login`)

Purpose: Authenticate municipal officials with role-based access

UI Elements:

- Email input
- Password input
- Login button
- Demo credentials hint

Demo Credentials:

```
MHO: mho@smc.gov / demo123
Epidemiologist: epi@smc.gov / demo123
Analyst: analyst@smc.gov / demo123
```

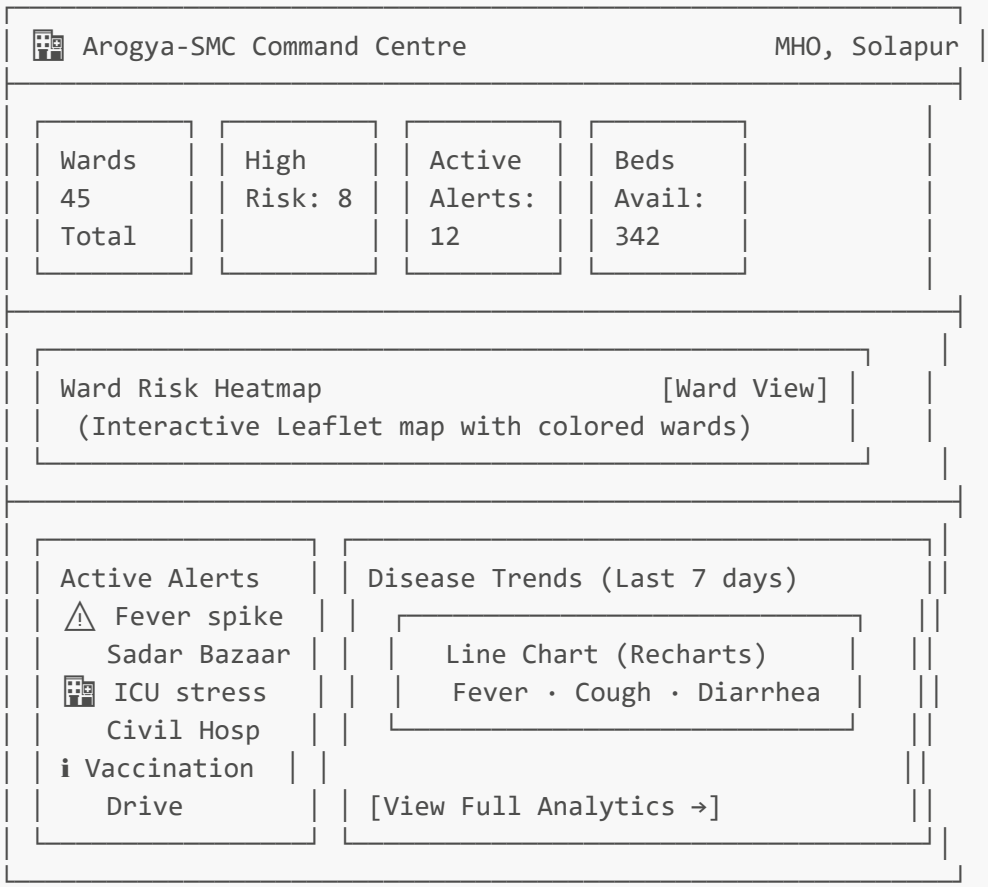
Logic:

- POST to `/api/auth/login`
- Receive JWT with role information
- Redirect to dashboard

Screen 2: Dashboard Overview (`/dashboard`)

Purpose: Command centre view – at-a-glance city health status

UI Layout:



Key Elements:

- **Header:** User info, last refresh timestamp
- **KPI Cards:** City-level metrics with trend indicators
- **Heatmap:** Interactive map showing ward risk levels (green/yellow/orange/red)
- **Alert Feed:** Latest 5 alerts with severity colors
- **Trend Chart:** Quick view of disease trends

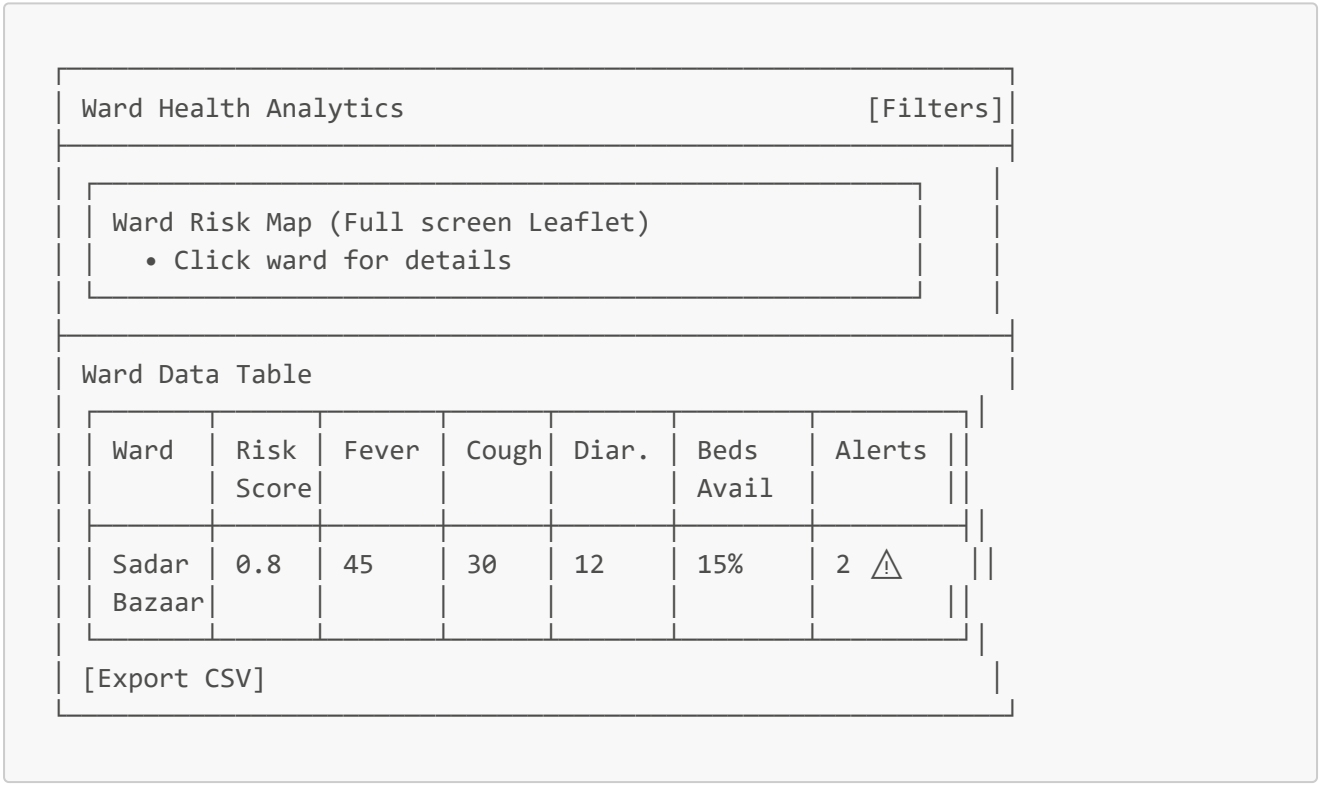
API Calls:

- `GET /api/dashboard/summary` → city stats + ward summaries
- `GET /api/dashboard/alerts?limit=5` → recent alerts
- `GET /api/dashboard/trends?days=7` → trend data

Screen 3: Ward Analytics (`/wards`)

Purpose: Detailed ward-level analysis

UI Layout:



API Calls:

- `GET /api/wards/geojson` → ward boundaries with risk scores
- `GET /api/wards/summary` → tabular data for all wards

Screen 4: Ward Detail (`/wards/[code]`)

Purpose: Drill-down into specific ward

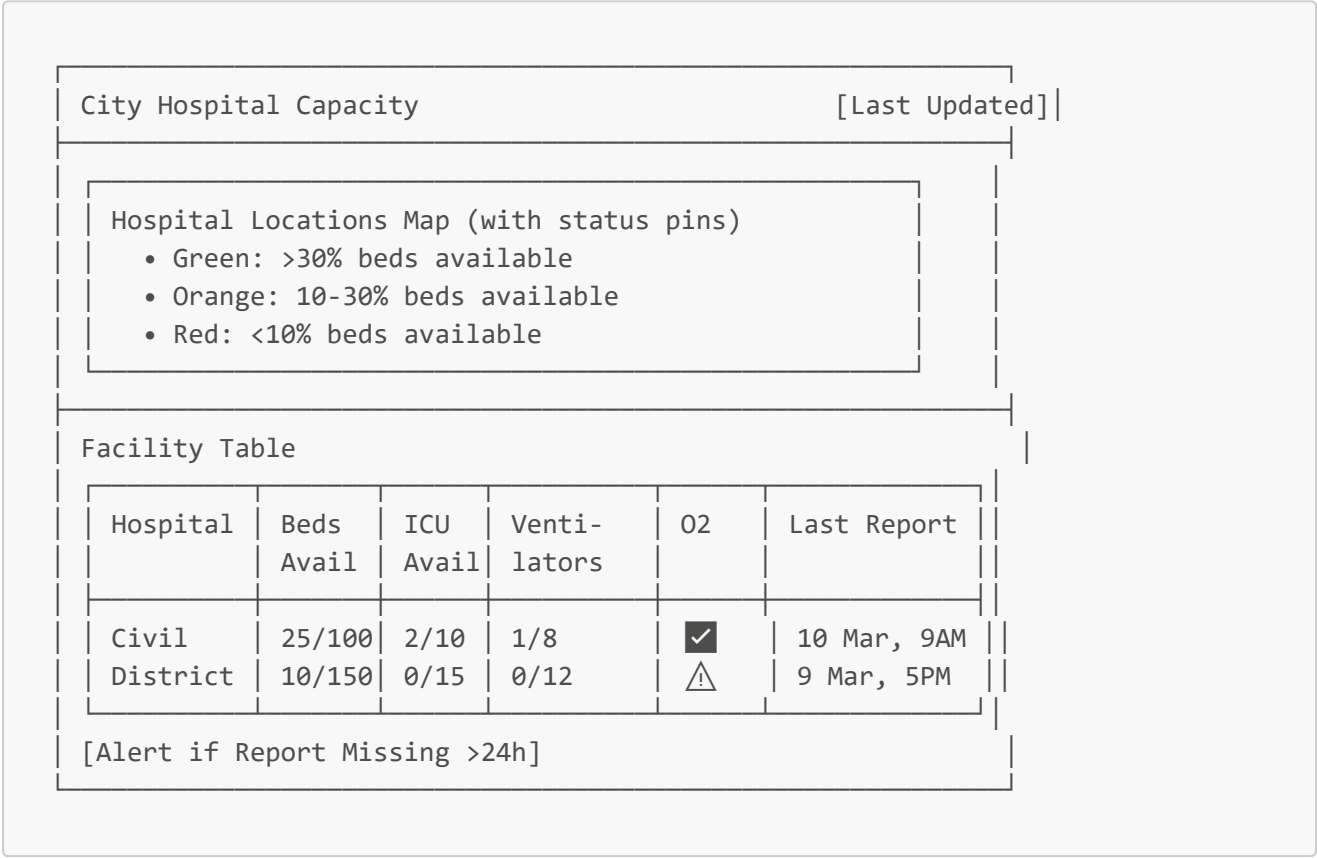
UI Elements:

- Ward header with name, risk score, population
- **Syndromic Trends:** 7-day and 30-day charts for all indicators
- **Facility List:** Hospitals in this ward with current capacity
- **Alert History:** Past alerts for this ward
- **ASHA Activity:** Recent reports from ASHA workers
- **Action Buttons:**
  - "Issue Ward-Specific Advisory"
  - "Notify ASHA Workers"
  - "Mark for Review"

Screen 5: Facility Monitor (`/facilities`)

Purpose: Real-time hospital capacity monitoring

UI Layout:



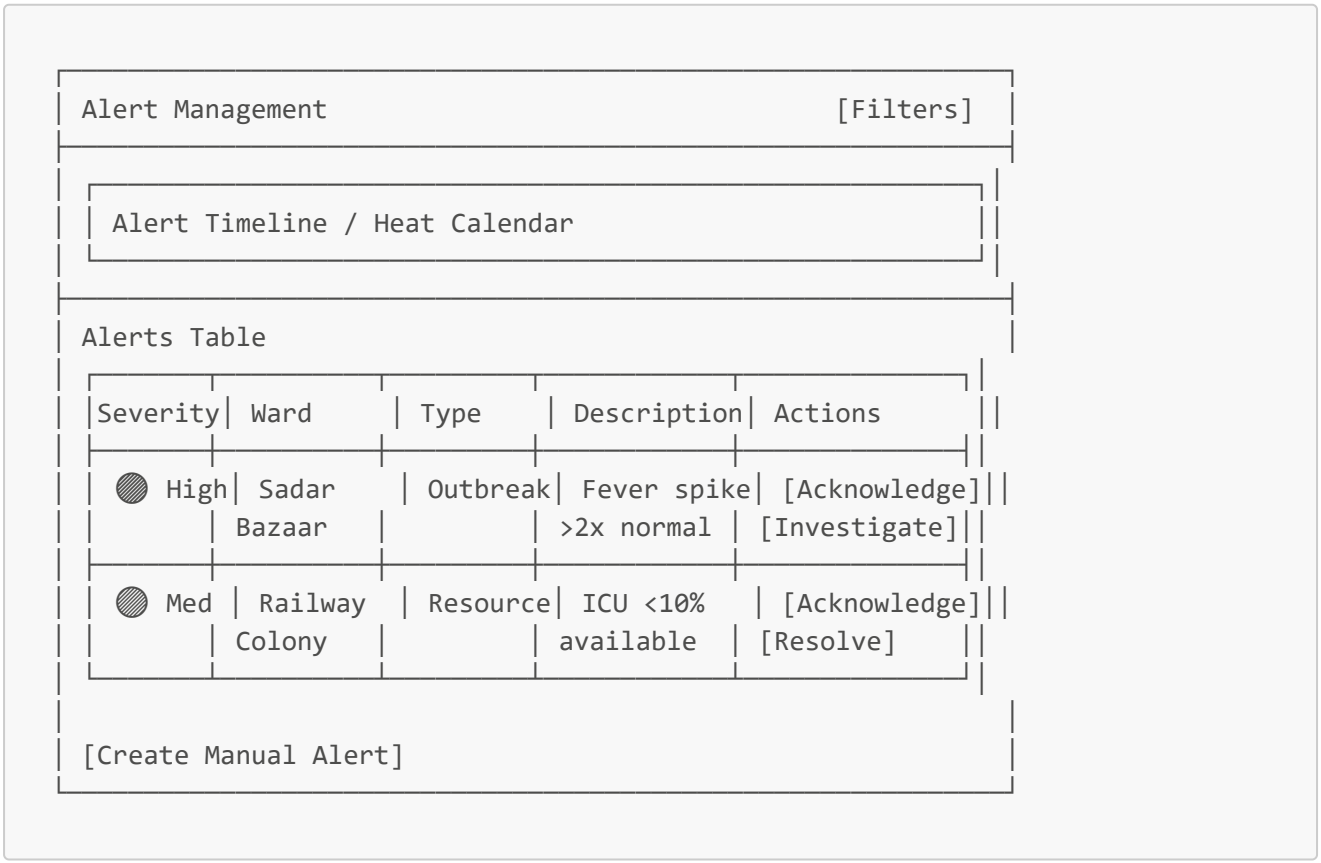
API Calls:

- `GET /api/facilities` → all facilities with latest capacity
- `GET /api/facilities/reporting-status` → which hospitals have reported today

Screen 6: Alert Management (`/alerts`)

Purpose: Monitor and respond to system-generated alerts

UI Layout:



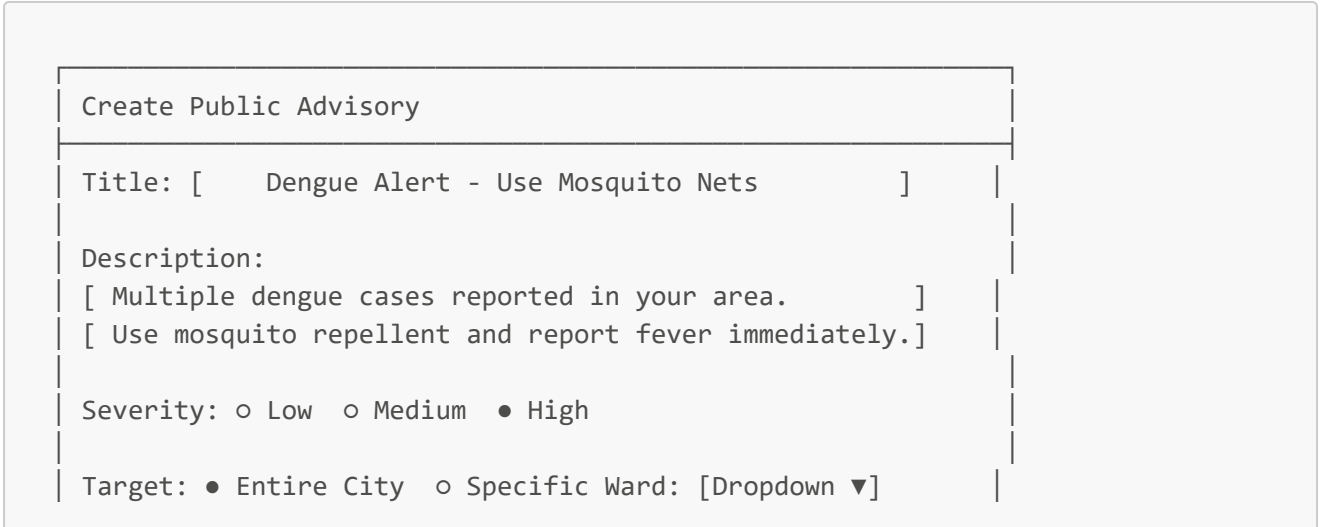
Alert Actions:

- **Acknowledge:** Assign to self, remove from "new" count
- **Investigate:** Open investigation workflow
- **Resolve:** Close alert with resolution notes
- **Escalate:** Change severity or notify higher authority

Screen 7: Advisory Creation (`/advisories/new`)

Purpose: Create and publish public health advisories

Form Design:



```
│                                                          │
│ Expiry: [   2026-03-17    ] (optional)                   │
│                                                          │
│ Languages:                                               │
│ ✓ English                                                │
│ ✓ Marathi (auto-translate)                               │
│ ✓ Hindi   (auto-translate)                               │
│                                                          │
│ [Preview]                          [Publish Advisory]    │
└──────────────────────────────────────────────────────────┘
```

Logic:

- On publish → POST to `/api/advisories`
- System creates multilingual versions
- FCM notifications sent to citizens in target ward
- Advisory appears in citizen app

Screen 8: Communication Module (`/communicate`)

Purpose: Send targeted messages to ASHA workers

UI Elements:

- **Recipient Selection:**
  - All ASHA workers
  - Ward-specific
  - Individual worker (by ID/name)
- **Message Type:**
  - Alert notification
  - Training reminder
  - General information
- **Message Content:** Title + body
- **Priority:** Normal / High (critical)
- **Schedule:** Send now / schedule for later

Example Use Case:
"Send training reminder to all ASHA workers in Sadar Bazaar about dengue prevention"

---

8. API Integration

Backend Endpoints

Endpoint	Method	Purpose
`/api/dashboard/summary`	GET	City-level KPIs and ward summaries
`/api/dashboard/trends`	GET	Time-series data for charts

Endpoint	Method	Purpose
`/api/dashboard/alerts`	GET	List alerts (with filters)
`/api/dashboard/alerts/:id`	PATCH	Update alert status
`/api/wards`	GET	List all wards
`/api/wards/geojson`	GET	GeoJSON for ward boundaries
`/api/wards/:code`	GET	Ward details
`/api/facilities`	GET	All facilities with capacity
`/api/facilities/:id`	GET	Single facility details
`/api/advisories`	GET/POST	List / create advisories
`/api/advisories/:id`	PUT/DELETE	Update / delete advisory
`/api/communicate`	POST	Send notifications

Real-time Updates

Using Server-Sent Events or WebSockets for live alert notifications:

```
// Client connects to SSE endpoint
const eventSource = new EventSource('/api/events');

eventSource.onmessage = (event) => {
  const data = JSON.parse(event.data);
  if (data.type === 'new_alert') {
    // Show notification, update alert count
    showToast(`New alert: ${data.alert.title}`);
    queryClient.invalidateQueries(['alerts']);
  }
};
```

---

9. Implementation Priority (13 Days)

Priority	Screens/Features	Day Allocation
P1 (Must Have)	Project setup, Login, Dashboard Overview, Ward Heatmap (static), Basic Alerts list	Days 1-4
P2 (Should Have)	Facility Monitor, Trend Charts, API integration, Alert management	Days 5-7
P3 (Nice to Have)	Advisory Creation, Communication module, Ward Detail view	Days 8-10

Priority	Screens/Features	Day Allocation
Buffer	Testing, bug fixes, polish, video recording	Days 11-13

Day-by-Day Breakdown

Day	Focus
1	Next.js project setup, Tailwind, shadcn/ui, folder structure
2	Login page, authentication, protected routes
3	Dashboard layout, KPI cards, static heatmap (sample GeoJSON)
4	Active alerts component, connect to mock API
5	Facility Monitor page, tables, status indicators
6	Trend charts (Recharts) with mock data
7	API integration – connect all components to backend
8	Alert management – acknowledge, resolve functionality
9	Advisory creation form
10	Communication module, user management (basic)
11	Testing across browsers, responsive design
12	Bug fixes, performance optimization
13	Final testing, video recording

10. Project Structure

```
smc-dashboard/
├── public/
├── src/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── page.tsx (redirect to dashboard)
│   │   ├── login/
│   │   │   └── page.tsx
│   │   ├── dashboard/
│   │   │   └── page.tsx
│   │   ├── wards/
│   │   │   ├── page.tsx
│   │   │   └── [code]/
│   │   │       └── page.tsx
│   │   ├── facilities/
│   │   │   └── page.tsx
```

```
│   │   ├── alerts/
│   │   │   └── page.tsx
│   │   ├── advisories/
│   │   │   ├── page.tsx
│   │   │   └── new/
│   │   │       └── page.tsx
│   │   ├── communicate/
│   │   │   └── page.tsx
│   │   └── api/ (Next.js API routes for mock backend)
│   ├── components/
│   │   ├── layout/
│   │   │   ├── Sidebar.tsx
│   │   │   └── Header.tsx
│   │   ├── dashboard/
│   │   │   ├── KPICards.tsx
│   │   │   ├── WardHeatmap.tsx
│   │   │   ├── AlertFeed.tsx
│   │   │   └── TrendChart.tsx
│   │   ├── wards/
│   │   │   ├── WardMap.tsx
│   │   │   └── WardTable.tsx
│   │   ├── facilities/
│   │   │   ├── FacilityMap.tsx
│   │   │   └── FacilityTable.tsx
│   │   ├── alerts/
│   │   │   └── AlertTable.tsx
│   │   └── ui/ (shadcn components)
│   ├── hooks/
│   │   ├── useDashboard.ts
│   │   ├── useAlerts.ts
│   │   └── useFacilities.ts
│   ├── lib/
│   │   ├── api/
│   │   │   └── client.ts
│   │   ├── auth/
│   │   │   └── auth.ts
│   │   └── utils/
│   │       ├── formatting.ts
│   │       └── constants.ts
│   ├── types/
│   │   └── index.ts
│   └── middleware.ts (auth)
├── .env.local
└── package.json
```

---

11. Success Criteria for Demo Video

The 4-5 minute demo should clearly show:

1. **Login** – MHO logs in with role-based access

2. **Dashboard Overview** – City-level KPIs, heatmap, alert feed
3. **Ward Drill-Down** – Click a ward to see detailed trends
4. **Facility Monitoring** – View hospital capacity across city
5. **Alert Management** – Acknowledge and resolve an alert
6. **Advisory Creation** – Create a public health advisory
7. **Communication** – Send notification to ASHA workers
8. **Real-time Update** – Show new alert appearing (simulated)

---

12. Key Design Principles

What to Emulate from Successful Systems

✅ **Role-based views** – Different users see relevant information
✅ **Real-time updates** – Live data without page refresh
✅ **Geospatial focus** – Maps make ward-level patterns immediately visible
✅ **Actionable alerts** – Each alert has clear next steps
✅ **Drill-down capability** – From city overview to ward detail
✅ **Export functionality** – Data can be shared in reports

What to Avoid

❌ **Information overload** – Prioritize key metrics, hide complexity
❌ **Slow performance** – Optimize database queries, use caching
❌ **Mobile-unfriendly** – Ensure dashboard works on tablets for field visits
❌ **Unclear alert ownership** – Every alert should be assignable

---

13. Conclusion

The Arogya-SMC Municipal Health Command Dashboard provides Solapur's health officials with the real-time intelligence they need to protect public health . By integrating data from ASHA workers, hospitals, and the citizen app into a unified geospatial interface, it enables:

- **Early outbreak detection** through automated alerts
- **Resource optimization** via real-time facility monitoring
- **Targeted communication** with field staff and citizens
- **Evidence-based policy** through trend analysis and reporting

With the 13-day implementation plan focused on core features, the prototype will demonstrate the complete command-and-control capabilities that make Arogya-SMC a transformative solution for Solapur Municipal Corporation.

Good luck with the implementation! 🏛️